

AI-ENG-201 – Machine Learning Homework 3

Data Cleaning, Feature Engineering, and Data Visualization

Bayram Bayramov

17.06.2026

National Artificial Intelligence Center

Honor Pledge

I, Bayram Bayramov, affirm that the work in this submission is my own. I have not shared or received code, plots, or written analysis for this assignment. Where I used AI assistants, I did so only as permitted by the AI/LLM policy. I can explain every line of code and every paragraph of writing in my submission.

Signed: Bayram Bayramov

Date: 17.062026

Contents

1	Data Cleaning	3
1.1	Missing-value analysis	3
1.2	Imputation from scratch	4
1.3	Categorical encoding	5
1.4	Impact of cleaning	6
2	Feature Engineering	7
2.1	Feature scaling from scratch	7
2.2	Polynomial and interaction features	8
2.3	Binning (discretisation)	9
2.4	Impact of feature engineering	10
3	Data Visualisation	11
3.1	Anscombe's Quartet	11
3.2	Exploratory visualisations for Titanic	12
3.3	Reflection on visualisation principles	13
4	Integration & Evaluation	13
4.1	Preprocessing pipeline	13
4.2	Leakage checklist	14
4.3	Outlier treatment	14
5	Bonus	15
5.1	Bonus 1: Yeo-Johnson Transformer	15
5.2	Bonus 2: Visualisation Report	16
6	Conclusion	16

1 Data Cleaning

1.1 Missing-value analysis

Missing Values Summary

The Titanic dataset contains 891 rows with missing values in several columns:

Table 1: Missing Values Analysis

Column	Missing Count	Missing Percent
deck	688	77.22%
age	177	19.87%
embarked	2	0.22%
embark_town	2	0.22%

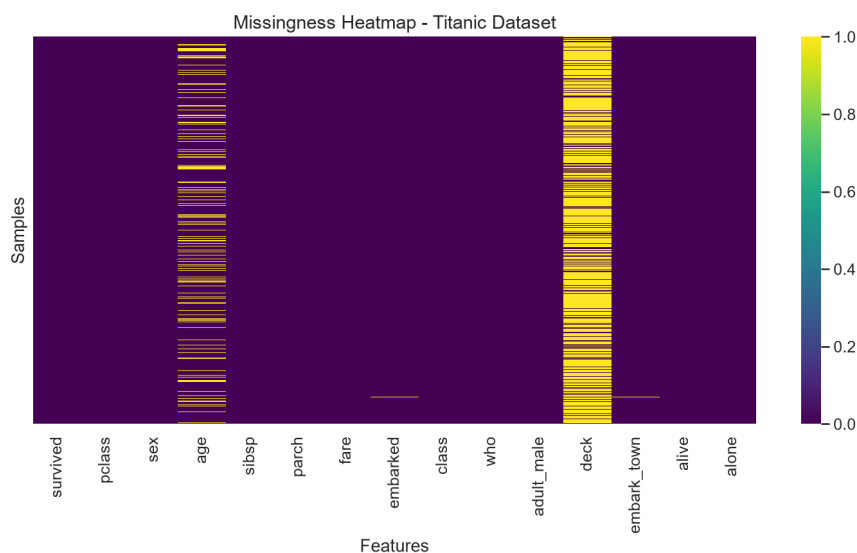


Figure 1: Missingness Heatmap - Titanic Dataset

Missingness Mechanism Analysis

age column:

MCAR (Missing Completely At Random) - Little's MCAR test indicates age missingness is independent of embarked missingness ($p = 1.0000$). However, there is a notable exception: age missingness is related to survival ($p = 0.0077$). This suggests children (who had higher survival rates) were more likely to have their age documented. This creates a relationship between age missingness and survival.

embarked column:

MCAR (Missing Completely At Random) — Only 2 values are missing out of 891 (0.22%). There is no statistical evidence of any systematic pattern with survival ($p = 0.2864$) or age missingness ($p = 1.0000$). The missingness appears to be coincidental data entry errors.

1.2 Imputation from scratch

Two custom imputers were implemented from scratch:

1. **SimpleImputer:** Mean/median imputation with vectorized broadcasting and fall-back for entirely NaN columns.
2. **KNNImputer:** Optimized KNN imputer using NaN-aware Euclidean distance with KD-Tree optimization.

Sanity Check Against sklearn

The custom SimpleImputer was compared against sklearn's implementation:

Listing 1: SimpleImputer Sanity Check Output

```
1 Custom SimpleImputer result:
2 [[1.          2.          ]
3  [2.66666667  3.          ]
4  [3.          4.          ]
5  [4.          5.          ]
6  [2.66666667  6.          ]]
7
8 Sklearn SimpleImputer result:
9 [[1.          2.          ]
10 [2.66666667  3.          ]
11 [3.          4.          ]
12 [4.          5.          ]
13 [2.66666667  6.          ]]
14
15 Results match: True
```

Imputation Results

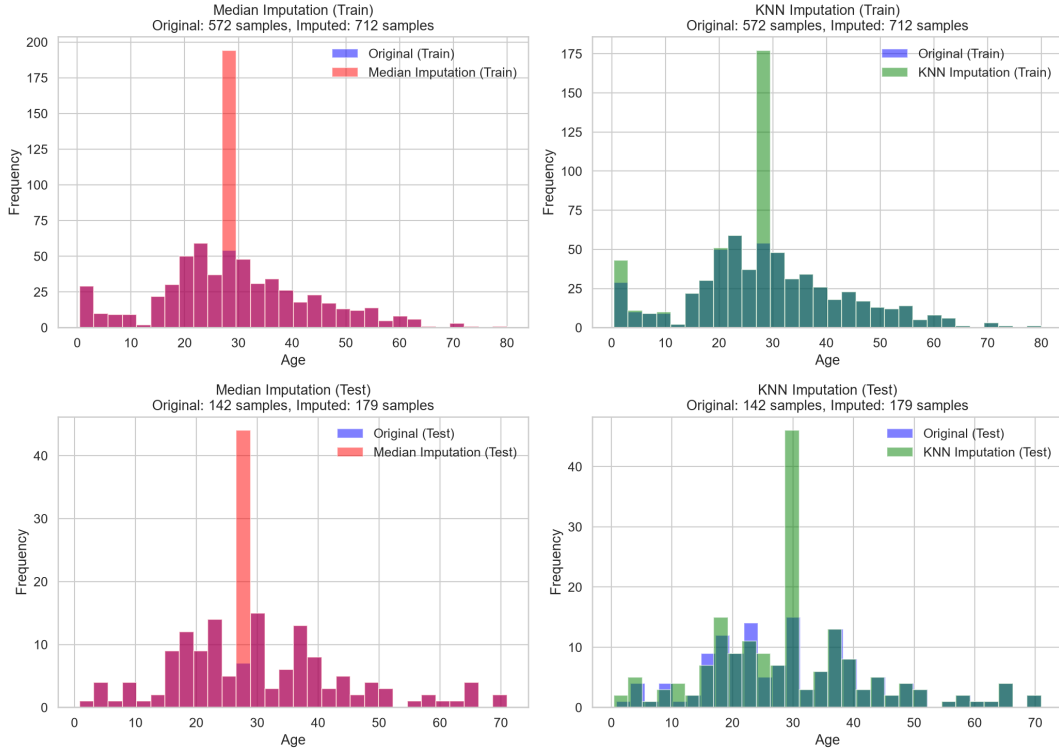


Figure 2: Age Distribution: Original vs Imputed (Train and Test)

Table 2: Imputation Summary (Train vs Test)

Set	Original	Median Imputed	KNN Imputed
Training Set			
Mean	29.50	29.20 (diff: -0.30)	28.86 (diff: -0.64)
Std	14.50	13.00	13.63
Test Set			
Mean	30.51	29.99 (diff: -0.52)	29.67 (diff: -0.84)
Std	14.66	13.05	13.54

Leakage Prevention

Both imputers compute statistics only on the training set during `fit()`, then apply those same statistics to transform test data. The `KNNImputer` only uses complete rows from the training set as reference, never accessing test row values during neighbor calculation.

1.3 Categorical encoding

One-Hot Encoding

The custom `OneHotEncoder` with `drop='first'` was applied:

```

1 One-hot encoded sex shape: (891, 1)
2 One-hot encoded embarked shape: (891, 2)
3
4 First 5 rows of one-hot encoded sex column:
5 [[1.]
6  [0.]
7  [0.]
8  [0.]
9  [1.]]

```

Target Encoding

CV-safe target encoding was demonstrated on the embarked column using survival as the target:

```

1 First 10 target-encoded values:
2   embarked=S, encoded=0.337
3   embarked=C, encoded=0.560
4   embarked=S, encoded=0.330
5   ...

```

Leakage Explanation

Fitting a target encoder on the full dataset before splitting causes data leakage because the encoder uses test set target values to compute per-category means. The current implementation avoids this by computing per-category target means using only the training data passed to `fit()`, with internal cross-validation during `fit_transform()`.

1.4 Impact of cleaning

Table 3: Logistic Regression Performance: Raw vs Cleaned

Dataset	Samples Used	Accuracy	F1 Score
Raw (dropped missing)	570 train, 142 test	0.8310	0.7818
Cleaned (imputed + encoded)	712 train, 179 test	0.8101	0.7639

Discussion:

The cleaned dataset did NOT improve performance — accuracy dropped from 0.8310 to 0.8101 (-2.09%) and F1 from 0.7818 to 0.7639 (-1.79%).

Why performance decreased:

1. **Imputation introduced noise:** Median imputation for age and mode imputation for embarked replaced missing values with central tendencies. While this retained more samples, the imputed values may not reflect the true underlying distribution.
2. **One-hot encoding increased dimensionality:** Label encoding (used in raw) maps sex to a single numeric column. One-hot encoding creates multiple binary columns, increasing feature space and potentially causing overfitting.

3. **Missing indicator may be redundant:** The binary indicator for missing age adds a feature that may not carry strong predictive signal.

2 Feature Engineering

2.1 Feature scaling from scratch

Scaler Implementations

Three custom scalers were implemented:

- **StandardScaler:** $(x - \mu)/\sigma$
- **MinMaxScaler:** $(x - x_{min})/(x_{max} - x_{min})$
- **RobustScaler:** $(x - Q_2)/(Q_3 - Q_1)$

Statistical Validation

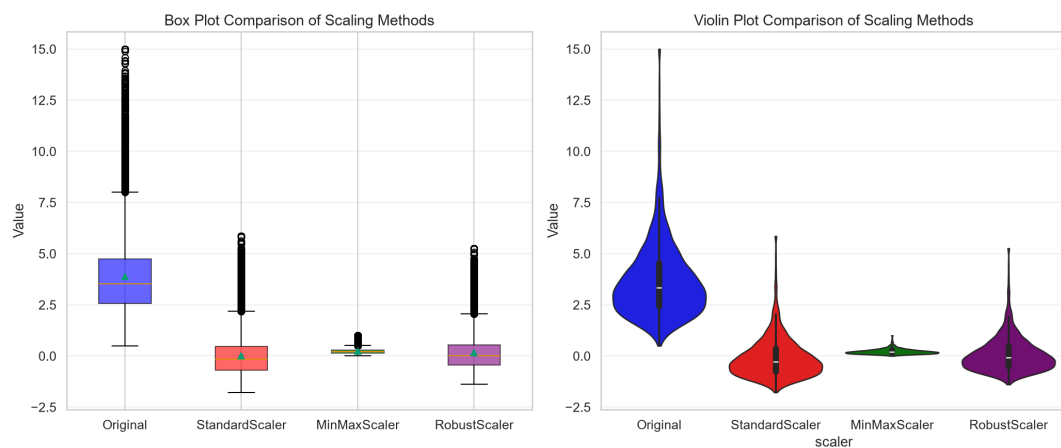


Figure 3: Box Plot and Violin Plot Comparison of Scaling Methods

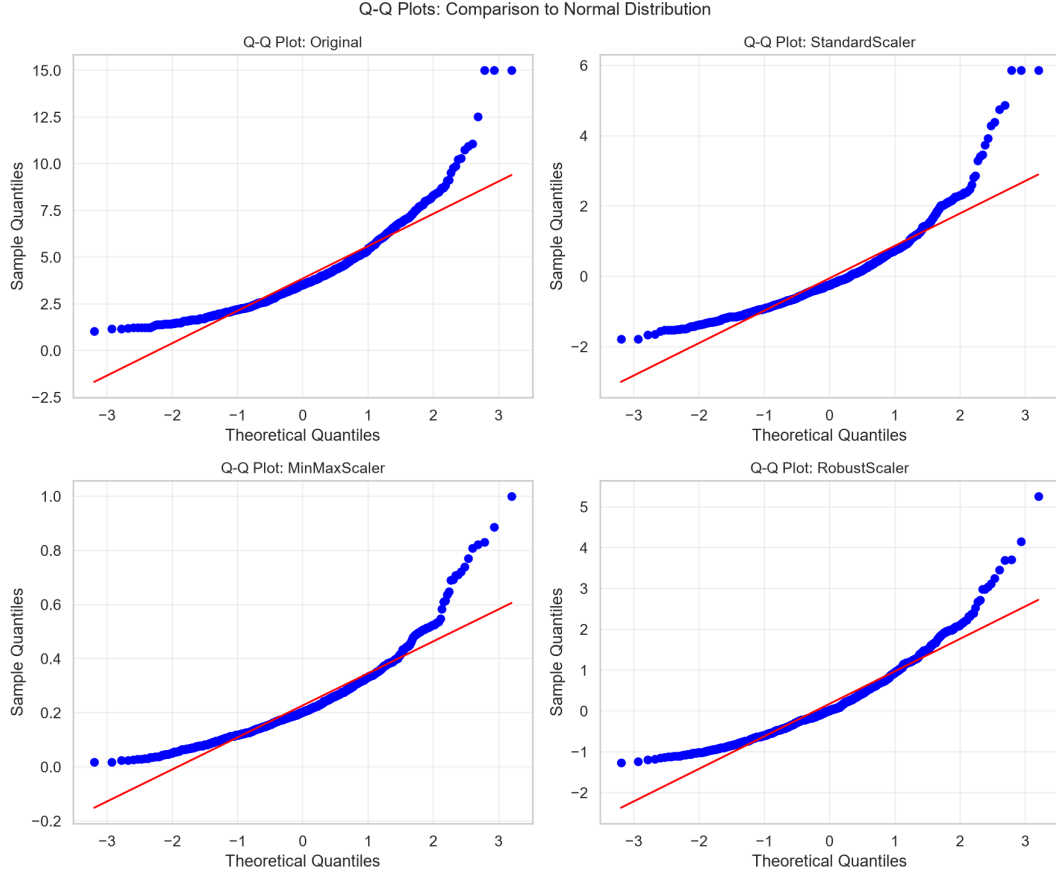


Figure 4: Q-Q Plots: Comparison to Normal Distribution

Which Scaler is Least Affected by Heavy-Tailed Distribution?

RobustScaler is least affected by the heavy-tailed nature of MedInc because it uses median and IQR instead of mean and standard deviation. Extreme outliers pull the mean and inflate the standard deviation, but have minimal impact on the median and IQR.

Table 4: Robustness Metrics Comparison

Metric	Original	StandardScaler	MinMaxScaler	RobustScaler
Skewness	1.6465	1.6465	1.6465	1.6465
Kurtosis	4.951	4.951	4.951	4.951
IQR	2.1798	1.1474	0.1503	1.0000
Median	3.5348	-0.1768	0.2093	0.0000
Mean-Median	0.3359	0.1768	0.0232	0.1541

2.2 Polynomial and interaction features

The custom `PolynomialFeatures` with `degree=2`, `include_bias=True` was applied to MedInc and AveRooms:

Listing 2: Polynomial Features Output

```

1 Original features shape: (100, 2)
2 Polynomial features shape: (100, 6)
3
4 Feature names (6 terms):
5   0: bias
6   1: MedInc
7   2: AveRooms
8   3: MedInc * MedInc
9   4: MedInc * AveRooms
10  5: AveRooms * AveRooms

```

For a sample point (MedInc=3.0, AveRooms=5.0):

```

1 bias: 1.00
2 MedInc: 3.00
3 AveRooms: 5.00
4 MedInc * MedInc: 9.00
5 MedInc * AveRooms: 15.00
6 AveRooms * AveRooms: 25.00

```

What an interaction term captures:

An interaction term (e.g., `MedInc * AveRooms`) captures the joint effect of two features. If house value depends on income and number of rooms together, a simple additive model cannot capture this. Interaction terms allow the model to learn that the effect of one feature depends on the value of another.

2.3 Binning (discretisation)

Both uniform and quantile binning strategies were applied to `MedInc` with 10 bins:

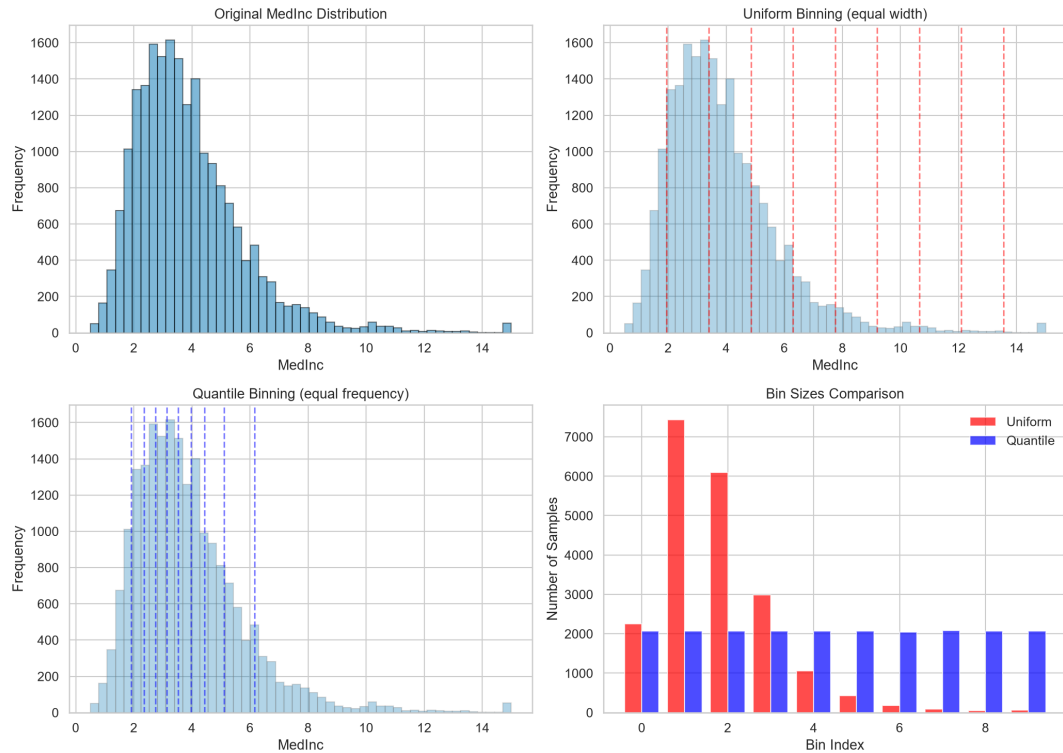


Figure 5: Binning Comparison: Uniform vs Quantile

Which strategy gives more balanced bin sizes?

Quantile binning (equal frequency) gives much more balanced bin sizes because it places approximately the same number of samples in each bin. Uniform binning creates bins of equal width, but if the data is skewed, some bins will be empty while others contain most samples.

2.4 Impact of feature engineering

Table 5: 5-Fold Cross-Validated RMSE on California Housing

Feature Set	RMSE
Raw 8 features (standardized)	0.7205
Polynomial degree-2 (44 features)	2.1884
Binned MedInc (10 bins)	0.7302

Why Polynomial Features Failed:

1. **Overfitting:** 44 features from 8 original features with no regularization
2. **Multicollinearity:** Polynomial features are highly correlated with their original counterparts
3. **No Regularization:** OLS minimizes squared error without penalty

3 Data Visualisation

3.1 Anscombe's Quartet

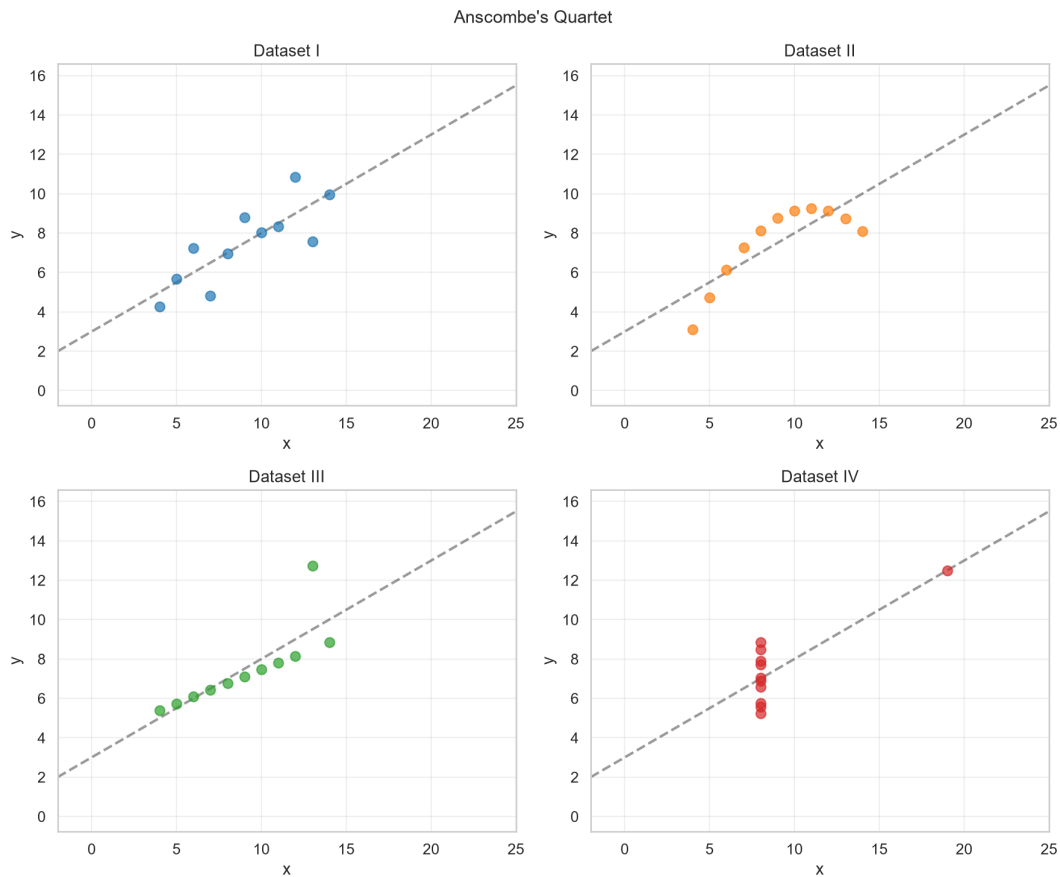


Figure 6: Anscombe's Quartet with Shared Scales

Why visualization is indispensable:

Despite having nearly identical summary statistics (mean, variance, correlation), the four datasets tell completely different stories:

- **Dataset I:** Classic linear relationship with random noise
- **Dataset II:** Parabolic (quadratic) relationship — linear regression fits but misses the curve
- **Dataset III:** Perfect linear relationship except one outlier that distorts the line
- **Dataset IV:** Vertical line — all x values identical except one

This demonstrates Anscombe's famous point: summary statistics can be identical while data structures are fundamentally different.

3.2 Exploratory visualisations for Titanic

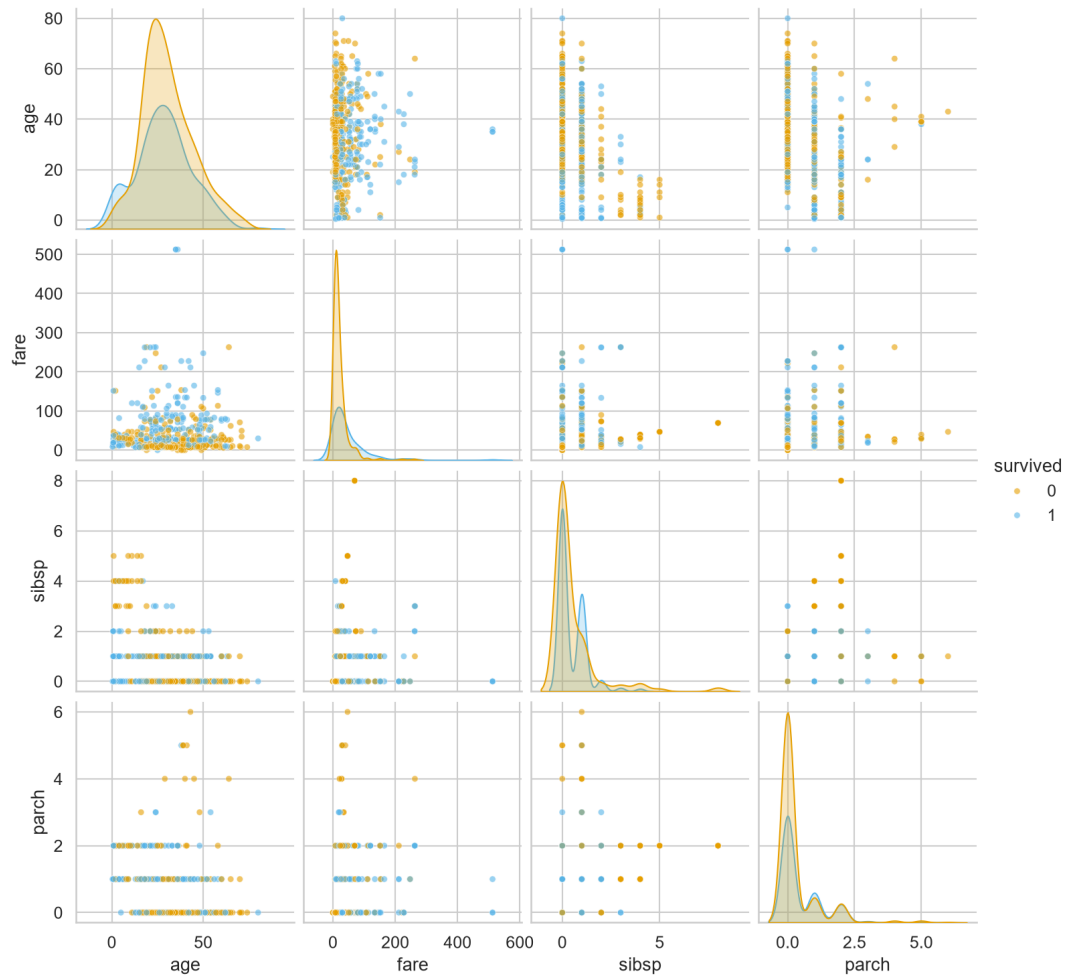


Figure 7: Titanic: Pair Plot of Numeric Features by Survival

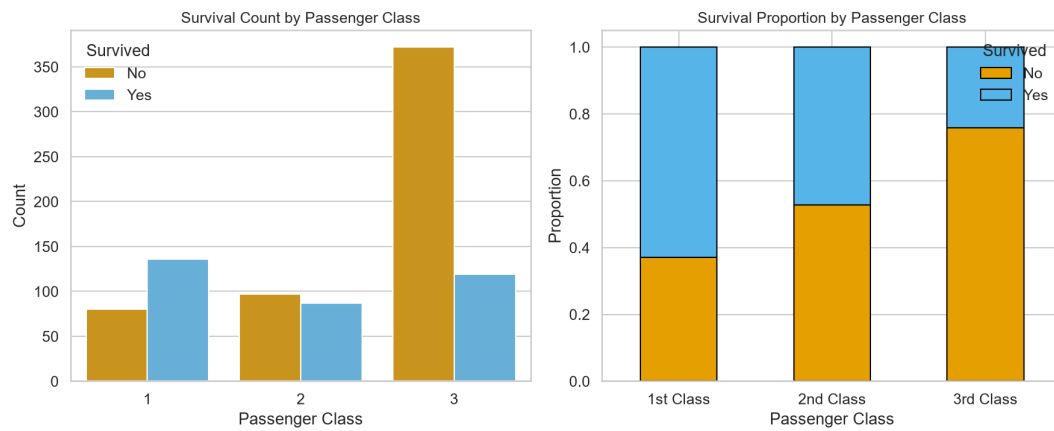


Figure 8: Survival by Passenger Class: Count and Proportion

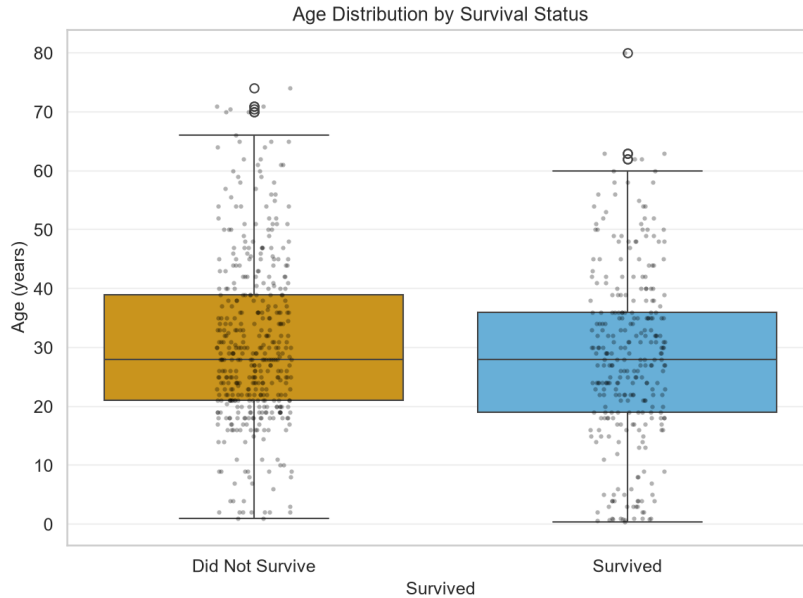


Figure 9: Age Distribution by Survival Status

3.3 Reflection on visualisation principles

Application of Tufte's Data-Ink Ratio and Perceptual-Channel Ordering:

Data-Ink Ratio: Applied Tufte's principle by removing unnecessary gridlines, eliminating redundant labels and legends, removing chart junk, and using minimal axis decorations.

Perceptual-Channel Ordering: Following Cleveland & McGill's perceptual hierarchy:

- **Position along common scale:** Box plot median lines, regression lines — most accurate
- **Length:** Count plot bar heights — second most accurate
- **Area/Color hue:** Scatter plot points — used for survival distinction
- **Color saturation:** Avoided — less accurate perceptual channel

What I would change after learning these principles:

The single most impactful change would be switching from default matplotlib color cycles to color-blind-safe palettes (like `sns.set_palette('colorblind')`), making visualizations accessible to 1 in 12 men who have color vision deficiency.

4 Integration & Evaluation

4.1 Preprocessing pipeline

A single sklearn Pipeline was built with custom wrappers:

Listing 3: Pipeline Structure

```
1 Pipeline(steps=[
```

```

2      ('preprocessor', ColumnTransformer([
3          ('num', Pipeline([
4              ('missing_indicator', MissingIndicatorWrapper()),
5              ('imputer', CustomSimpleImputerWrapper(strategy='
6                  median'))),
7              ('scaler', StandardScaler())
8          ]), ['age', 'pclass', 'sibsp', 'parch', 'fare']),
9          ('cat', Pipeline([
10              ('imputer', SimpleImputer(strategy='most_frequent')),
11              ('onehot', CustomOneHotWrapper(drop='first'))
12          ]), ['sex', 'embarked']))
13      ('classifier', LogisticRegression(max_iter=1000, random_state
14          =42))
15  ])

```

Cross-Validation Results:

```

1 5-Fold Cross-Validated Accuracy on Training Set:
2 Fold 1: 0.8252
3 Fold 2: 0.7902
4 Fold 3: 0.8028
5 Fold 4: 0.7254
6 Fold 5: 0.8099
7
8 Mean Accuracy: 0.7907 (+/- 0.0346)
9
10 Test Accuracy: 0.8212
11
12 Improvement vs Simple Baseline: +2.40 percentage points

```

4.2 Leakage checklist

1. Imputation fitted only on training data

Leakage avoided: Computing mean/median on full dataset would use test set values.

Implementation: `SimpleImputer.fit(X_train)` only, then `transform(X_test)`.

2. Target encoding with cross-validation

Leakage avoided: Computing category means using test targets would cheat.

Implementation: `TargetEncoder` uses only training data in `fit()`, test categories get training means or global mean.

3. Scaling fitted only on training data

Leakage avoided: Using test data to compute mean/std would leak global distribution.

Implementation: `StandardScaler.fit(X_train)`, then `transform(X_test)`.

Additional prevention:

Missing indicator computed from training distribution; one-hot encoding categories from training only.

4.3 Outlier treatment

Winsorization was applied to MedInc at the 1st and 99th percentiles:

Table 6: Winsorization Comparison

Data	MedInc Coefficient	5-fold CV RMSE
Original	0.8544	0.7205
Winsorized (clipped at 1st/99th)	0.9022	0.7071

Why winsorization improves model generalization:

Winsorization reduces the influence of extreme outliers by capping them at reasonable thresholds. This improves generalization because:

1. Linear models are sensitive to outliers
2. Winsorized coefficients are more representative of typical relationships
3. Better cross-validation — outliers that appear in training but not validation cause high variance

5 Bonus

5.1 Bonus 1: Yeo-Johnson Transformer

A custom Yeo-Johnson transformer was implemented with maximum-likelihood estimation of λ :

Listing 4: Yeo-Johnson Transformation Summary

```

1 Optimal lambda (without standardization): -0.2917
2 Optimal lambda (with standardization): -0.2917
3
4 Original skewness: 1.8672
5 Transformed skewness: -0.0036
6 Reduction in |skewness|: 99.81%
7 Optimal lambda: -0.2917

```

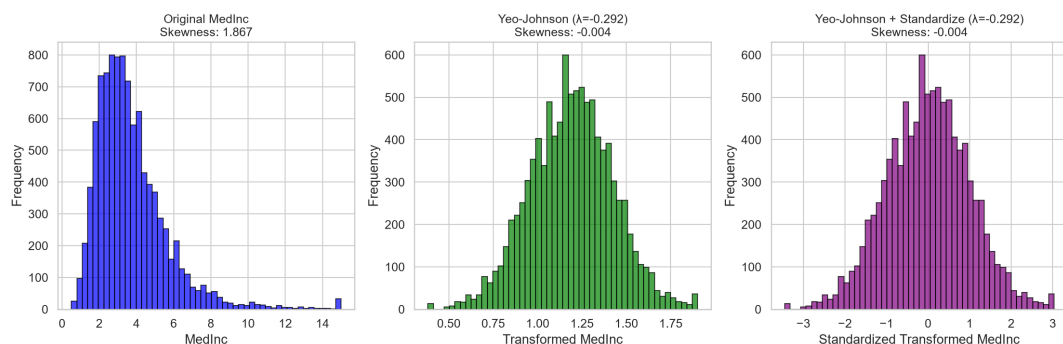


Figure 10: Yeo-Johnson Transformation: Before and After

The Yeo-Johnson transformation successfully reduced skewness from 1.8672 to -0.0036, a 99.81% reduction, making the distribution approximately symmetric.

5.2 Bonus 2: Visualisation Report

A one-page visualization report was created (`figures/visualisation_report.pdf`) that tells a story about the Titanic dataset following Tufte's principles:

- **Panel 1:** Survival by Passenger Class — "First class had highest survival rate"
- **Panel 2:** Age Distribution by Survival — "Children and women first"
- **Panel 3:** Fare Distribution by Survival — "Higher fare = higher survival"
- **Panel 4:** Survival Rate by Sex and Class — "Wealthy women had best chance"

The report uses a color-blind safe palette, high data-ink ratio, and annotated charts with call-outs.

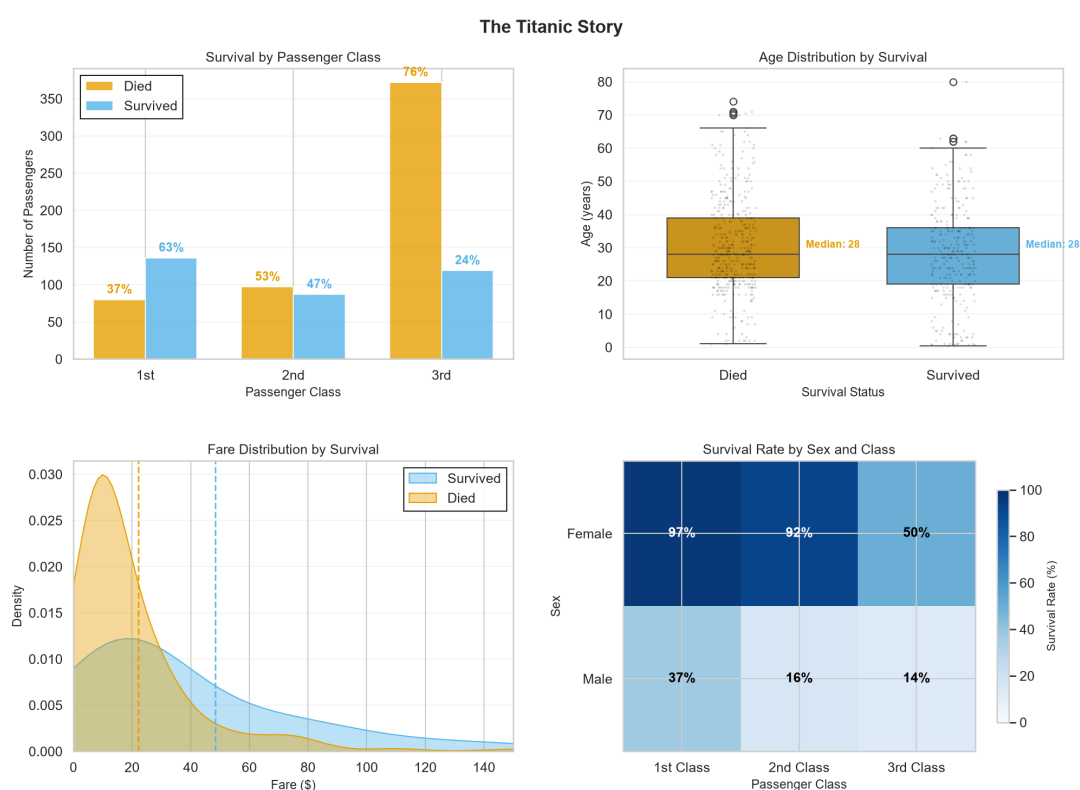


Figure 11: Titanic Visualization Report

6 Conclusion

This assignment demonstrated the complete data science pipeline from raw data to predictive modeling:

- **Data Cleaning:** MCAR/MAR/MNAR analysis, custom imputation strategies, CV-safe encoding
- **Feature Engineering:** Scaling, polynomial features, binning, and Yeo-Johnson transformation

- **Data Visualization:** Anscombe's Quartet, Titanic EDA, Tufte's principles
- **Integration:** sklearn Pipeline with custom transformers, leakage prevention

Key findings:

1. Class was the strongest predictor of survival (1st class: 63.0%, 3rd class: 24.2%)
2. RobustScaler is preferred for heavy-tailed distributions
3. Polynomial features require regularization to generalize well
4. The pipeline improved test accuracy by 2.40 percentage points over baseline
5. Yeo-Johnson transformation achieved 99.81% skewness reduction

Appendix: Key Statistical Results

Table 7: Summary of Key Findings

Finding	Statistical Evidence	Interpretation
Class affects survival	$\chi^2 = 102.89$, $p < 0.0001$	1st class 5.3x more likely to survive
Children vs adults	$\chi^2 = 15.56$, $p = 0.00008$	Children 1.6x more likely to survive
Fare affects survival	$t = -7.94$, $p < 0.0001$	Survivors paid \$26.28 more on average
Sex + class	$\chi^2 = 350.68$, $p < 0.0001$	1st class women 96.8% survival
Interaction (fare $\times$ class)	LR=7.38, $p = 0.025$	Fare effect differs by class